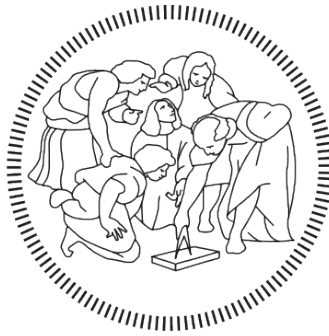


Space and time adaptivity solver

Giacomo Maglio - 10858201

Francesca Morciano - 10900578

Giuseppe Hares - 11081817



POLITECNICO
MILANO 1863

July 2026

Contents

1	Introduction	3
2	Mathematical model	4
2.1	Governing equation	4
2.2	Boundary and Initial Conditions	4
2.2.1	Neumann Boundary	4
2.2.2	Initial Condition	4
2.3	Forcing term	4
3	Numerical method	6
3.1	Weak Formulation	6
3.2	Spatial Discretization	6
3.3	Time Discretization and Algebraic Solver	6
3.4	Space-Time Adaptive Strategy	7
3.4.1	Temporal Advance and Error Estimation	7
3.4.2	Spatial Error Estimation and Hanging Nodes	8
3.5	Deal II Implementation details	9
3.5.1	Triangulation Class	9
3.5.2	Solver implementation	9
3.5.3	Run Loop	10
4	Numerical experiments	11
4.1	Setup	11
4.2	Adaptive solution results	12
4.3	Error analysis	13
5	Discussion	14
5.1	Scalability	15
5.2	Limitations	17
6	Conclusion	18
7	References	19

1 Introduction

Partial differential equations play a fundamental role in the mathematical modeling of physical phenomena such as heat transfer, fluid dynamics, and wave propagation. Among them, the heat equation is a classical diffusion problem that describes the evolution of temperature in a medium over time. Although its mathematical formulation is relatively simple, accurately solving the heat equation can become computationally demanding when the solution exhibits localized features or rapidly changing dynamics.

In many practical applications, the numerical resolution required to accurately approximate the solution is not uniform throughout the computational domain or over the entire simulation time. Regions characterized by steep gradients or localized sources require a finer spatial discretization, while smoother regions can be represented using a coarser mesh without significantly affecting the solution accuracy. Similarly, periods of rapid temporal evolution benefit from smaller time steps, whereas larger time steps are sufficient when the solution changes slowly. Employing a uniform discretization in both space and time therefore leads to unnecessary computational costs, as high resolution is maintained even where it is not needed.

Adaptive numerical methods address this issue by dynamically adjusting the spatial mesh and the time-step size according to suitable error indicators. This strategy concentrates computational resources only where and when they are required, improving the balance between numerical accuracy and computational efficiency. Such approaches are particularly relevant in the context of high-performance scientific computing, where reducing the number of degrees of freedom and minimizing execution time are key objectives.

The purpose of this project is to develop a space and time adaptive finite element solver for the heat equation using the deal.II library. The forcing term is defined as the product of a spatial Gaussian distribution and a sequence of localized temporal impulses, producing a solution characterized by highly localized transient phenomena. The adaptive solver is implemented following the methodologies presented in the deal.II tutorials and is compared against a solver employing a homogeneous discretization in terms of solution accuracy and computational cost.

The remainder of this report is organized as follows. Chapter 2 introduces the mathematical formulation of the problem. Chapter 3 presents the finite element discretization together with the adaptive space-time strategy and the main implementation details. Chapter 4 illustrates the numerical experiments and compares the adaptive solver with a homogeneous discretization. Chapter 5 discusses the obtained results, including computational performance and scalability considerations, while Chapter 6 summarizes the main conclusions and possible future developments.

2 Mathematical model

2.1 Governing equation

The problem considered in this project is the heat equation with a time-dependent localized source term.

The governing equation is defined as

$$\frac{\partial u}{\partial t} - \nabla \cdot (\mu \nabla u) = f \quad \text{in } \Omega \times (0, T),$$

where $u(\mathbf{x}, t)$ denotes the temperature field, Ω is the computational domain, T is the final simulation time, and μ is the diffusion coefficient.

Throughout this work, the diffusion coefficient is assumed to be constant and equal to one,

$$\mu = 1.$$

Since the diffusion coefficient is uniform, the governing equation simplifies to

$$\frac{\partial u}{\partial t} - \Delta u = f,$$

where Δ denotes the Laplace operator.

The heat equation models the diffusion of thermal energy within the domain, while the source term f represents an external heat input that varies in both space and time.

2.2 Boundary and Initial Conditions

2.2.1 Neumann Boundary

The equation is complemented by homogeneous Neumann boundary conditions,

$$\nabla u \cdot \mathbf{n} = 0 \quad \text{on } \partial\Omega \times (0, T),$$

where $\mathbf{n}$ denotes the outward unit normal vector to the boundary.

These boundary conditions impose a zero heat flux through the boundary, meaning that no thermal energy enters or leaves the computational domain across its boundaries.

2.2.2 Initial Condition

The initial condition is

$$u(\mathbf{x}, 0) = 0 \quad \text{in } \Omega,$$

which corresponds to an initially uniform temperature distribution before the localized heat source becomes active.

2.3 Forcing term

A distinctive feature of this problem is the forcing term, which is constructed as the product of independent temporal and spatial functions,

$$f(\mathbf{x}, t) = g(t)h(\mathbf{x}).$$

- The temporal component is defined as

$$g(t) = \frac{\exp(-a \cos(2N\pi t))}{\exp(a)},$$

where the parameter $a > 0$ controls the sharpness of the temporal impulses, while the integer N determines the number of excitation cycles occurring during the simulation.

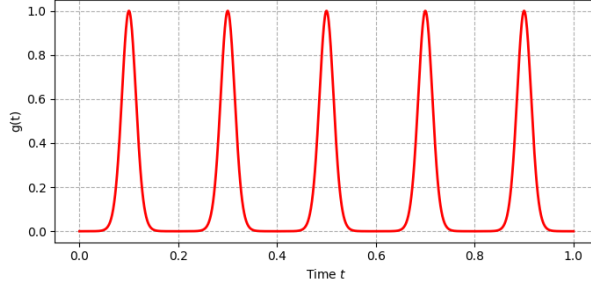


Figure 1: Temporal Forcing Term $g(t)$ for $a = 5, N = 5$.

- The spatial component is given by the Gaussian function

$$h(\mathbf{x}) = \exp\left(-\frac{|\mathbf{x} - \mathbf{x}_0|^2}{\sigma^2}\right),$$

where $\mathbf{x}_0$ identifies the center of the heat source and σ controls its spatial spread. The Gaussian distribution ensures that the source is highly localized around $\mathbf{x}_0$, with its intensity rapidly decreasing away from the center.

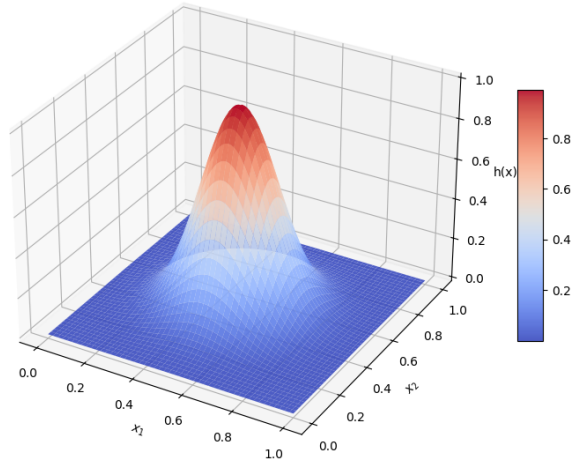


Figure 2: Spatial Forcing Term $h(\mathbf{x})$ for $\sigma = \frac{1}{5}, (\mathbf{x}_{1,0}, \mathbf{x}_{2,0}) = (\frac{1}{2}, \frac{1}{2})$.

The localized nature of the forcing term produces solution features that evolve unevenly in both space and time. Consequently, adaptive numerical techniques are expected to provide a more efficient discretization than uniform meshes and constant time steps. These characteristics motivate the use of adaptive finite element methods, whose formulation is presented in the following chapter.

3 Numerical method

3.1 Weak Formulation

Let $V = H^1(\Omega)$ denote the trial and test function space. Multiplying the governing equation by a test function $v \in V$ and integrating over the computational domain yields:

$$\int_{\Omega} \frac{\partial u}{\partial t} v d\Omega - \int_{\Omega} \nabla \cdot (\mu \nabla u) v d\Omega = \int_{\Omega} f v d\Omega \quad (1)$$

$$\int_{\Omega} \frac{\partial u}{\partial t} v d\Omega + \int_{\Omega} \mu \nabla u \cdot \nabla v d\Omega - \int_{\partial\Omega} (\mu \nabla u \cdot \mathbf{n}) v d\gamma = \int_{\Omega} f v d\Omega \quad (2)$$

Since homogeneous Neumann boundary conditions are imposed ($\mu \nabla u \cdot \mathbf{n} = 0$), the boundary integral vanishes.

Defining $F(v) = \int_{\Omega} f v d\Omega$ and the bilinear form $a(u, v) = \int_{\Omega} \mu \nabla u \cdot \nabla v d\Omega$, the weak formulation reads:

$$\forall t \in [0, T], \text{ find } u \in V \text{ such that } \begin{cases} \int_{\Omega} \frac{\partial u}{\partial t} v d\Omega + a(u, v) = F(v) & \forall v \in V \\ u|_{t=0} = 0 \end{cases}$$

3.2 Spatial Discretization

For the spatial approximation, we employ the Galerkin method. We introduce a finite-dimensional subspace $V_h \subset V$ spanned by a set of basis functions $\{\varphi_j\}_{j=1}^{N_h}$. The discrete solution is expressed as a linear combination $u_h(x, t) = \sum_{j=1}^{N_h} u_j(t) \varphi_j(x)$.

In the implementation, the finite-dimensional space V_h is constructed using continuous Lagrange finite elements of degree r .

By restricting the test functions to $v_h = \varphi_i \in V_h$ and substituting the expansion of u_h , the semi-discrete problem is reduced to a system of Ordinary Differential Equations (ODEs):

$$M \frac{d\vec{u}(t)}{dt} + A\vec{u}(t) = \vec{F}(t)$$

where $\vec{u}(t)$ is the vector of unknown nodal values, $M_{ij} = \int_{\Omega} \varphi_i \varphi_j d\Omega$ is the mass matrix, $A_{ij} = a(\varphi_j, \varphi_i)$ is the stiffness matrix, and $F_i(t) = F(\varphi_i)$.

The integrals required to assemble the system matrices are evaluated by Gaussian quadrature of order $r+1$, which guarantees an accurate numerical integration of the finite element operators.

3.3 Time Discretization and Algebraic Solver

To fully discretize the ODE system

$$M \frac{d\vec{u}(t)}{dt} + A\vec{u}(t) = \vec{F}(t),$$

we introduce a partition of the time interval $0 = t_0 < t_1 < \dots < t_N = T$, with a variable

step size $\Delta t_n = t_{n+1} - t_n$. Applying the general θ -method yields the fully discrete algebraic problem:

$$M \frac{\vec{u}^{n+1} - \vec{u}^n}{\Delta t_n} + \theta A \vec{u}^{n+1} + (1 - \theta) A \vec{u}^n = \theta \vec{F}^{n+1} + (1 - \theta) \vec{F}^n$$

Rearranging the terms for the unknown vector $\vec{u}^{n+1}$, we obtain the linear system:

$$\left(\frac{1}{\Delta t_n} M + \theta A \right) \vec{u}^{n+1} = \left(\frac{1}{\Delta t_n} M - (1 - \theta) A \right) \vec{u}^n + \theta \vec{F}^{n+1} + (1 - \theta) \vec{F}^n$$

In our numerical framework, we focus on the **Crank-Nicolson scheme** due to its higher temporal accuracy. In the present implementation, the value ($\theta = 0.5$) is specified in the main driver of the program, making Crank-Nicolson the default integration scheme adopted in all numerical experiments.

From a theoretical standpoint, selecting $\theta \geq 0.5$ is a fundamental prerequisite for time-adaptive algorithms, as these schemes are **unconditionally stable**. This ensures that the choice of Δt_n is dictated solely by accuracy requirements, avoiding restrictive conditions typical of explicit methods ($\theta = 0$), which would drastically degrade performance during spatial refinement.

At every time step, the resulting sparse linear system is solved using the Conjugate Gradient method preconditioned by Algebraic Multigrid (AMG), exploiting the symmetry and positive definiteness of the system matrix.

3.4 Space-Time Adaptive Strategy

Our solver combines spatial and temporal adaptivity, allowing both the grid and the time step to automatically adjust to localized changes in the solution. At each time step, this adaptation is carried out using a sequential operator-splitting method.

3.4.1 Temporal Advance and Error Estimation

Inside the while loop in the **AdaptiveHeat::run** method, the current solution u_h^n is stored before computing a new one, as well as Δt_n is, in case of a rollback scenario. At the beginning of step $n + 1$, the system is advanced using the current time step Δt_n to find a tentative solution u_h^{n+1} . To track the temporal discretization error without solving a computationally expensive dual problem, we adopt a localized physical criterion based on the norm of the variation of the solution:

$$\delta U = \|u_h^{n+1} - u_h^n\|_\infty$$

The infinity norm is inexpensive to compute and provides a practical indicator of the magnitude of the transient evolution between two consecutive time levels.

The value δU serves as a proxy for the local truncation error. We then compute an adaptation factor:

$$\text{factor} = \max \left(0.3, \min \left(\frac{\text{tol}}{\delta U}, 2.0 \right) \right)$$

If $\delta U > \text{tol}$, the current step is rejected (*Adaptive Rollback*): the time is reset, the solution is reverted to u_h^n , and the step is re-attempted with a penalized $\Delta t_{\text{new}} = 0.9 \cdot \Delta t_n \cdot \text{factor}$.

If $\delta U \leq \text{tol}$, the step is accepted.

This rollback mechanism guarantees that every accepted time step satisfies the prescribed tolerance while avoiding the propagation of inaccurate solutions.

3.4.2 Spatial Error Estimation and Hanging Nodes

Once a temporal step is successfully accepted, the spatial error distribution is evaluated using the `KellyErrorEstimator` class, provided by deal.II [1].

This estimator is residual-based and approximates the local contribution to the error by measuring the jump of the normal flux across interior faces of each active cell:

$$\eta_K^2 \approx h_K \int_{\partial K} [\mu \nabla u_h \cdot n]^2 d\gamma,$$

where $[\mu \nabla u_h \cdot n]$ denotes the jump of the normal component of the diffusive flux across cell interfaces.

Since the computation is performed on a distributed mesh, the local error indicators are combined across all MPI processes through a global reduction operation. The resulting global error estimate is then used to determine the refinement and coarsening fractions consistently over the entire computational domain.

Cells are then selected for refinement and coarsening using a fixed-fraction strategy applied to the distribution of the local error indicators. In particular we chose the refine and coarsening fractions based on the ratio between the estimated error and spatial tolerance. Unlike a classical fixed-fraction strategy, the refinement and coarsening percentages are dynamically adjusted according to the ratio between the estimated global error and the prescribed spatial tolerance. Consequently, the mesh adaptation becomes more aggressive when the estimated error is large and more conservative as the solution approaches the desired accuracy.

A critical aspect of spatial adaptivity in deal.II is the treatment of non-conforming mesh interfaces generated by adaptive refinement. When a cell is refined but its neighbor is not, hanging nodes are created [4]. Continuity across these interfaces is enforced through constraint equations, assembled using `DoFTools::make_hanging_node_constraints`. These constraints are stored separately and are applied during the assembly and solution stages to ensure a globally conforming finite element solution.

After mesh adaptation, the numerical solution is transferred onto the new mesh using the deal.II `SolutionTransfer` class before redistributing the degrees of freedom and rebuilding the system matrices. This is done to avoid finding oneself having to integrate over shape functions defined on two different meshes, as described in [2].

3.4.2.1 Algorithmic Stability and Space-Time Separation

A key feature of the time-stepping controller is the inclusion of a **0.9 safety factor** in the update rule:

$$\Delta t_{n+1} = 0.9 \cdot \Delta t_n \cdot \text{factor}$$

The necessity of this factor is strictly mathematical and related to the dead-beat control of ODE solvers. Since the error proxy δU is an a posteriori estimation based on the current

step, expanding Δt to its strict theoretical limit ($\text{tol}/\delta U$) would result in systematic rejections at the subsequent step due to local variations in the source function $g(t)$. The 0.9 factor acts as a conservative damping buffer to ensure smooth transitions in the time-step evolution.

Furthermore, to prevent non-linear feedback loops between spatial remeshing and temporal adaptivity, the algorithm treats spatial refinement discretely. Rather than artificially coupling the grid size h and the time step via a hardcoded reduction (e.g., halving Δt upon refinement), the two structures communicate dynamically. When `refine_grid` changes the spatial resolution, the sharper gradients are naturally captured by the next temporal iteration; if these gradients generate a sharp transient, the physical proxy δU will automatically trigger a localized reduction of Δt .

To avoid unstable chattering between space and time operators, a dead-band threshold is enforced via the condition

```
timestep_number - last_refine_step >= min_steps_between_refine,
```

guaranteeing that the spatial configuration remains locked for a minimum number of steps to let the algebraic solver stabilize.

3.5 Deal II Implementation details

3.5.1 Triangulation Class

In the `AdaptiveHeat::setup` class, the initial domain is partitioned using hexahedral elements via `GridGenerator::hyper_cube`. The functional space V_h is constructed using standard continuous Galerkin finite elements `FE_Q<dim>(r)`, with integration performed via Gaussian quadrature `QGauss<dim>(r+1)`.

To ensure scalability, we used classes that support distributed-memory parallelization via MPI, adopting some of the ideas described in [3]. In particular we used `parallel::distributed::Triangulation<dim>` for the triangulation; we chose not to use `parallel::fullydistributed::Triangulation` because, even if it is more scalable (it does not replicate coarse mesh in all MPI nodes), it is newer and does not support some methods used in the solver.

3.5.2 Solver implementation

Since the resulting system matrix $\left(\frac{1}{\Delta t_n}M + \theta A\right)$ is symmetric and positive definite, the linear system is solved at each time step in `AdaptiveHeat::solve_time_step` using the Conjugate Gradient(CG) method. To guarantee mesh-independent convergence rates, the `deal.II` CG solver is preconditioned using an Algebraic Multigrid(AMG) method (`TrilinosWrappers::PreconditionAMG`).

Since the implementation relies on distributed Trilinos matrices and vectors, the solver naturally operates in parallel over the MPI partition of the computational mesh.

3.5.3 Run Loop

Algorithm 1 Adaptive space-time solution procedure

```
Initialize mesh and finite element space
while  $t < T$  do
  Assemble the linear system
  Solve the linear system
  Estimate the temporal error  $\delta_U$ 
  if  $\delta_U > \text{tol}_t$  then
    Restore the previous solution (rollback)
    Reduce the time step  $\Delta t$ 
    Continue with the same time step
  else
    Update the time step using the safety factor
    if mesh adaptation is triggered then
      Estimate the spatial error using the Kelly estimator
      Refine and/or coarsen the mesh
      Transfer the solution to the new mesh
      Redistribute the degrees of freedom
      Reassemble the system matrices
    end if
    Write the output files
    Advance the simulation time
  end if
end while
```

The solution of the problem is carried out by the **AdaptiveHeat::run** method. Within its body, initial conditions are applied first, by interpolating the u_0 function onto the mesh. Then, we enter into a while loop that advances in time. Within the loop, at each iteration, we assemble and solve the problem for one time step. Then, if the temporal error estimation is above tolerance, we perform timestep adaptivity until we reach an acceptable solution. After that, we perform spatial refinement, but only after a number of steps to avoid performing it too often since it is computationally expensive. Finally we write the obtained solution to a **.vtk** file, to later explore the solution with **Paraview**.

4 Numerical experiments

4.1 Setup

To study the performance of our solver, inside the `main.c` file, a flag `COMPARE_WITH_BASE` can be set to compare the adaptive solver to a baseline homogeneous solver. Both the solvers use the same parameters, the baseline solver uses a high refinement both for the mesh and timestep.

We used the following parameters:

Parameter	Value
Ω	$[0, 1]^3$
θ	0.5
$Initial\Delta t$	0.05
a	1.5
N	3
σ	0.5
T	1
x_0	(0, 0, 0)

After both the baseline and the adaptive solvers are done running, the error between the two solutions is computed and the timing results for each solving step is shown.

The initial mesh is shown below in Figure 3:

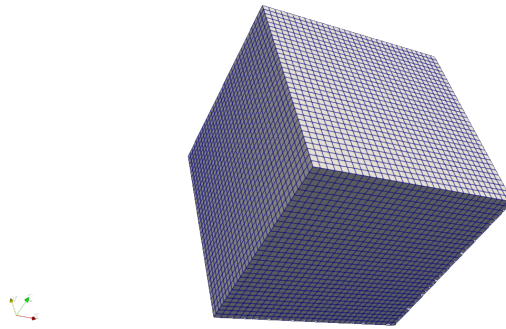


Figure 3: Initial mesh

4.2 Adaptive solution results

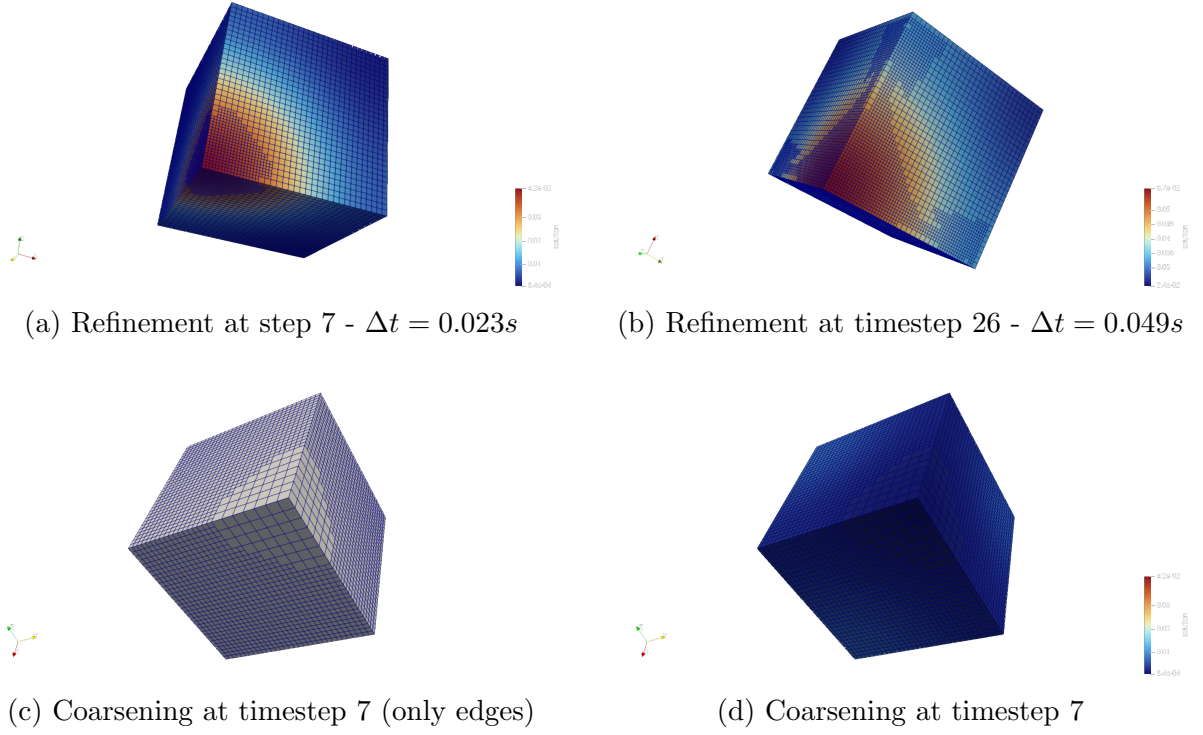


Figure 4: Evolution of the mesh at different timesteps.

From Figure 4 we can see that, where the solution changes swiftly, refinement is applied. In Figure 4 (c) and (d) we see the opposite side of the cube, showing that where the solution evolves more smoothly, coarsening is applied. While in Figure 5 we see the timestep getting smaller at the same time a burst of heat occurs.

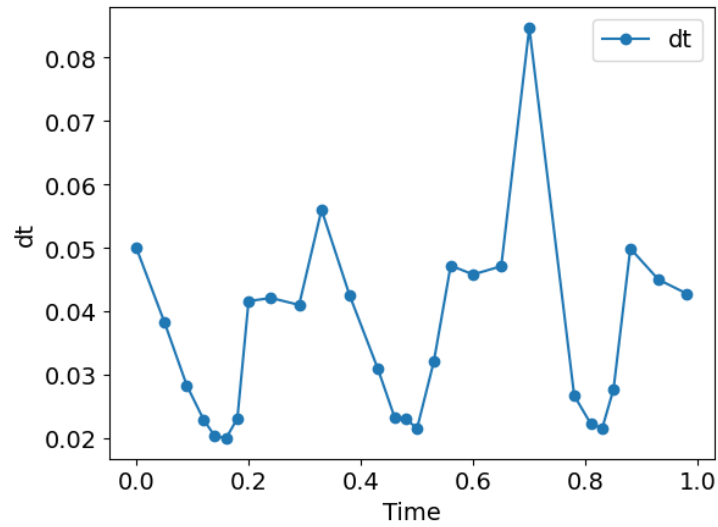
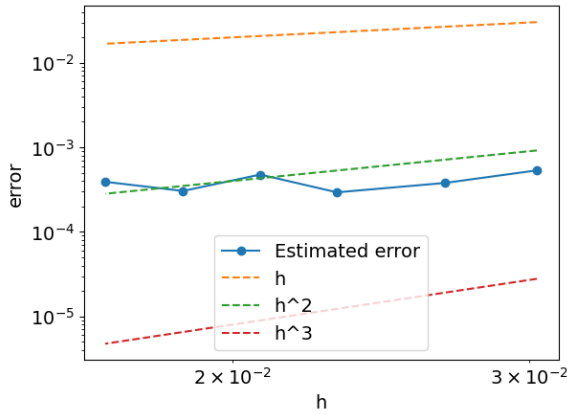


Figure 5: Timestep evolution in time

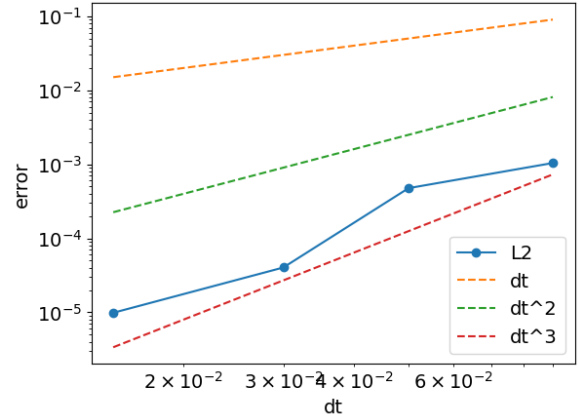
4.3 Error analysis

To study spatial convergence, we fixed Δt and used $h = \frac{1}{N_{el}}$. We estimated the error with `KellyEstimator` and plotted the results in Figure 6a.

Analogously, for temporal convergence, we fixed h and plotted the error $\|u_{\Delta t} - u_{\frac{\Delta t}{2}}\|_{L^2}$ in Figure 6b. The results obtained were not exactly aligned with our expectations of convergence being in the order of $O(\Delta t^p + h^q)$, with $p = 2$ when $\theta = 0.5$ and $q = 2$ for the L^2 norm [5]. However, this could be attributed to the estimations of the error used which, especially for temporal convergence, are not very accurate.



(a) Estimated spatial convergence



(b) Estimated temporal convergence

Quantity	Homogeneous solver	Adaptive solver
Spatial discretization	fixed mesh	adaptive mesh
Max Degrees of freedom	35937	82891
Number of time steps	40	42
Minimum time step [s]	0.025	0.02
Maximum time step [s]	0.025	0.0847
L^2 error against the baseline	-	$1.33e - 03$
CPU time [s]	15.85069	30.9324

5 Discussion

In this section we will show a comparison between a baseline homogeneous solver, chosen as reference, and the adaptive solver.

In particular the reference solver was compared against:

- Spatial adaptivity only
- Time adaptivity only
- Both space and time adaptivity

This was done to evaluate the impact of the two approaches independently and together.

Note: The `refine` and `time-adapt` timers are not used in the total count computation since each of them includes several other steps (like assembling and solving the system). However, it is still interesting, especially in the scalability study, to see how much time is spent in those two routines as a whole.

Moreover, all these tests were run on one MPI process.

Baseline solver

Timer	Calls	Min (s)	Avg (s)	Max (s)	Total (s)
update-ghost	40	1.17×10^{-4}	1.92×10^{-4}	4.22×10^{-4}	0.00769
solve	40	6.74×10^{-2}	8.87×10^{-2}	1.37×10^{-1}	3.55
output	40	7.12×10^{-2}	8.70×10^{-2}	1.39×10^{-1}	3.48
assemble	40	1.82×10^{-1}	2.13×10^{-1}	2.61×10^{-1}	8.50
setup	1	3.13×10^{-1}	3.13×10^{-1}	3.13×10^{-1}	0.313

Total Time	Max DoFs	Error against baseline
15.85	35 937	0.0

Space and Time

Timer	Calls	Min (s)	Avg (s)	Max (s)	Total (s)
refine	12	1.96×10^{-1}	3.56×10^{-1}	8.15×10^{-1}	4.28
time-adap	28	1.06×10^{-1}	1.41×10^{-1}	2.12×10^{-1}	3.95
solve	42	2.27×10^{-1}	3.12×10^{-1}	4.82×10^{-1}	13.1
output	28	1.06×10^{-1}	1.40×10^{-1}	2.11×10^{-1}	3.93
assemble	42	1.87×10^{-1}	3.26×10^{-1}	5.21×10^{-1}	13.7
update-ghost	73	1.10×10^{-5}	3.20×10^{-4}	1.20×10^{-3}	0.023
setup	1	1.79×10^{-1}	1.79×10^{-1}	1.79×10^{-1}	0.179

Total Time	Max DoFs	Error against baseline
30.93	82 891	1.33×10^{-3}

Space Only

Timer	Calls	Min (s)	Avg (s)	Max (s)	Total (s)
refine	5	2.21×10^{-1}	5.87×10^{-1}	1.04	2.93
time-adap	20	8.22×10^{-2}	1.60×10^{-1}	2.64×10^{-1}	3.20
solve	20	1.39×10^{-1}	3.44×10^{-1}	5.44×10^{-1}	6.89
output	20	8.19×10^{-2}	1.59×10^{-1}	2.62×10^{-1}	3.17
assemble	20	2.09×10^{-1}	3.70×10^{-1}	5.98×10^{-1}	7.40
update-ghost	43	1.39×10^{-5}	3.86×10^{-4}	1.42×10^{-3}	0.0166
setup	1	2.55×10^{-1}	2.55×10^{-1}	2.55×10^{-1}	0.255

Total Time	Max DoFs	Error against baseline
17.73	99 978	6.56×10^{-5}

Time Only

Timer	Calls	Min (s)	Avg (s)	Max (s)	Total (s)
time-adap	27	7.31×10^{-2}	8.56×10^{-2}	1.12×10^{-1}	2.31
solve	40	1.28×10^{-1}	1.92×10^{-1}	2.57×10^{-1}	7.69
output	27	7.27×10^{-2}	8.51×10^{-2}	1.12×10^{-1}	2.30
assemble	40	1.72×10^{-1}	2.07×10^{-1}	2.49×10^{-1}	8.27
update-ghost	67	5.00×10^{-5}	2.39×10^{-4}	6.81×10^{-4}	0.0160
setup	1	2.26×10^{-1}	2.26×10^{-1}	2.26×10^{-1}	0.226

Total Time	Max DoFs	Error against baseline
18.50	35 937	1.35×10^{-3}

The results indicate that spatial adaptivity has the greatest influence on the solution accuracy, while temporal adaptivity successfully controls the time discretization error by keeping it close to the prescribed tolerance. Moreover, the space and time adaptive solver incurs in a higher computational cost, indeed, because the number of times the system is solved is higher and the mesh is finer in some steps, solving the system is the biggest bottleneck; However, in the scalability section we will see that parallelization dramatically reduces these additional costs.

5.1 Scalability

Since the solver fully supports parallelization through **MPI**, we studied scalability. In particular we ran the space and time adaptive solver with 1, 2, 3 and 4 **MPI** processes. The results are shown in the tables below.

N = 1

Timer	Calls	Min (s)	Avg (s)	Max (s)	Total (s)
refine	12	2.32×10^{-1}	4.82×10^{-1}	9.60×10^{-1}	5.78
time-adap	28	8.43×10^{-2}	1.47×10^{-1}	2.38×10^{-1}	4.12
solve	42	1.74×10^{-1}	2.79×10^{-1}	4.09×10^{-1}	11.7
output	28	8.39×10^{-2}	1.46×10^{-1}	2.36×10^{-1}	4.10
assemble	42	1.83×10^{-1}	3.15×10^{-1}	5.50×10^{-1}	13.2
update-ghost	73	1.23×10^{-5}	3.10×10^{-4}	1.19×10^{-3}	0.0227
setup	1	2.05×10^{-1}	2.05×10^{-1}	2.05×10^{-1}	0.205

N = 2

Timer	Calls	Min (s)	Avg (s)	Max (s)	Total (s)
refine	24	1.50×10^{-1}	1.60×10^{-1}	7.26×10^{-1}	3.83
time-adap	56	5.64×10^{-2}	4.99×10^{-2}	1.74×10^{-1}	2.80
solve	84	1.56×10^{-1}	1.33×10^{-1}	5.21×10^{-1}	11.2
output	56	5.46×10^{-2}	4.90×10^{-2}	1.71×10^{-1}	2.74
assemble	84	9.72×10^{-2}	8.94×10^{-2}	2.82×10^{-1}	7.51
update-ghost	146	5.77×10^{-6}	7.48×10^{-4}	2.03×10^{-2}	0.109
setup	2	1.44×10^{-1}	7.22×10^{-2}	1.44×10^{-1}	0.144

N = 3

Timer	Calls	Min (s)	Avg (s)	Max (s)	Total (s)
refine	39	1.25×10^{-1}	8.52×10^{-2}	6.88×10^{-1}	3.32
time-adap	84	4.62×10^{-2}	2.62×10^{-2}	1.20×10^{-1}	2.20
solve	126	1.59×10^{-1}	8.63×10^{-2}	4.86×10^{-1}	10.9
output	84	4.45×10^{-2}	2.58×10^{-2}	1.17×10^{-1}	2.16
assemble	126	7.30×10^{-2}	4.57×10^{-2}	2.73×10^{-1}	5.76
update-ghost	219	3.52×10^{-6}	5.89×10^{-4}	1.93×10^{-2}	0.129
setup	3	1.56×10^{-1}	5.19×10^{-2}	1.56×10^{-1}	0.156

N = 4

Timer	Calls	Min (s)	Avg (s)	Max (s)	Total (s)
refine	56	9.35×10^{-2}	4.80×10^{-2}	4.93×10^{-1}	2.69
time-adap	112	4.27×10^{-2}	1.61×10^{-2}	9.24×10^{-2}	1.81
solve	168	1.20×10^{-1}	5.59×10^{-2}	3.70×10^{-1}	9.39
output	112	4.18×10^{-2}	1.58×10^{-2}	9.10×10^{-2}	1.77
assemble	168	6.56×10^{-2}	2.61×10^{-2}	1.71×10^{-1}	4.38
update-ghost	292	2.88×10^{-6}	3.97×10^{-4}	1.39×10^{-2}	0.116
setup	4	1.58×10^{-1}	3.94×10^{-2}	1.58×10^{-1}	0.158

These tables show that the impact of refinement is strongly mitigated by parallelization, making the adaptive solver a compelling alternative to the homogeneous solver. In particular, even if functions like `update-ghost` need to be executed more frequently, the speedup in other sections easily makes up for that latency.

5.2 Limitations

Despite the advantages, adaptive methods introduce additional computational overhead due to error estimation, mesh modification, and solution transfer. Moreover, the choice of tolerances influences the balance between accuracy and efficiency. The spatial and temporal tolerances selected for the present work, may require further tuning for different problem configurations.

6 Conclusion

Overall, the proposed adaptive strategy successfully combines spatial mesh refinement and adaptive time integration to efficiently resolve localized transient phenomena while reducing unnecessary computational effort. The numerical experiments demonstrate that adaptive discretizations provide a favorable balance between accuracy and computational cost compared with homogeneous meshes, provided that parallelization is available. Otherwise, the adaptive solver shows a higher CPU time compared to the baseline homogeneous solver which defeats the purpose of reducing computational cost.

Future work may include goal-oriented error estimators, adaptive polynomial enrichment, and a detailed parallel scalability study on larger distributed-memory systems.

7 References

References

- [1] Deal.II Documentation. Kelly error estimator class reference.
- [2] Deal.II Documentation. The step-26 tutorial program.
- [3] Deal.II Documentation. The step-40 tutorial program.
- [4] Deal.II Documentation. The step-6 tutorial program.
- [5] Politecnico di Milano Numerical Methods For Partial Differential Equations. Lecture notes, 2025/2026.